

Ansible Beginner Website Deployment Guide

Use one Linux computer to build and manage websites on two servers

ANSIBLE CONTROL 192.168.253.20 Sends the instructions	RHEL WEB 1 192.168.253.21 Receives website 1	RHEL FILE 1 192.168.253.22 Receives website 2
--	---	--

Audience Early high school students and nontechnical beginners

Outcome Two working web pages deployed from one control computer

What You Will Learn

In this lab, you will use one computer to send a reusable set of instructions to two Linux servers. Those instructions will install web-server software, create a different web page for each server, start the service, and allow web traffic through the firewall.

The visible result is simple: two browser tabs, two server addresses, and two working websites. The important lesson is that one approved automation file can configure many computers consistently.

The Big Idea

Imagine a school with two classrooms. Without automation, a technician must visit each room and repeat the same work. With Ansible, the technician writes one work order and sends it securely to both rooms from a central office.

Technical term	Everyday meaning
Ansible control node	The central management computer
Managed server	A computer receiving instructions
Inventory	An address book of computers
YAML playbook	A written work order
SSH	A secure delivery route
Module	A tool that performs one job
Play recap	A completion report
Idempotency	Avoiding work that is already finished

What Success Looks Like

- Both managed servers answer Ansible with ping: pong.
- The playbook finishes with unreachable=0 and failed=0.
- The first website identifies rhel-web1.
- The second website identifies rhel-file1.
- A second playbook run reports changed=0.

Phase 1 Prepare the Three Computers

Create three Linux virtual machines in VirtualBox. Each VM uses a private host-only network for the lab. A NAT adapter is also useful because it gives each VM internet access for software downloads.

Computer	Private address	Purpose
ansible-control	192.168.253.20	Stores the project and runs Ansible
rhel-web1	192.168.253.21	Managed server and website 1
rhel-file1	192.168.253.22	Managed server and website 2

VirtualBox Network Settings

- Adapter 1 uses NAT for internet and package repositories.
- Adapter 2 uses Host-only Adapter for private communication between the lab systems.
- Cable Connected must be checked for both adapters.

Important: The 192.168.253.0 network is specific to this lab. A different VirtualBox host-only network may use another address range.

Check Each Computer

```
hostname
ip -br address
```

A hostname is the computer's name. An IP address is its network location. It is similar to a person having both a name and a home address.

Phase 2 Prepare the Server Accounts

Ansible needs an approved account that can enter each managed server and perform administrative work. Complete this phase on both rhel-web1 and rhel-file1.

Create the Account

```
sudo useradd -m -G wheel administrator
```

If the account already exists, add it to the wheel group instead:

```
sudo usermod -aG wheel administrator
```

Set the Password

```
sudo passwd administrator
```

Enable Secure Remote Access

```
sudo systemctl enable --now sshd
```

Why Wheel Matters

On Rocky Linux and RHEL, members of the wheel group are normally permitted to use sudo. Sudo allows an approved user to perform an administrative task without signing in directly as the all-powerful root account.

Checkpoint

```
id administrator
systemctl is-active sshd
```

The account information should include wheel, and the SSH service should report active.

Phase 3 Test Communication

Perform the following tests from ansible-control. These checks prove that the central computer can reach both managed servers.

Test the Network

```
ping -c 3 192.168.253.21  
ping -c 3 192.168.253.22
```

Test Secure Login

```
ssh administrator@192.168.253.21  
hostname  
sudo -v  
exit  
ssh administrator@192.168.253.22  
hostname  
sudo -v  
exit
```

What Is Happening

Ping checks whether a computer can be reached across the network. SSH goes further by opening an encrypted remote session. Ansible uses SSH as its secure delivery route.

Expected result: The first SSH session reports rhel-web1. The second reports rhel-file1.

Phase 4 Create an SSH Key

An SSH key acts like an electronic badge. The control node uses its private key to prove its identity. Only the matching public key is placed on each managed server.

Generate the Key

```
ssh-keygen -t ed25519 -f ~/.ssh/rhel_lab_ed25519 -C "ansible-lab-key"
```

Copy the Public Key

```
ssh-copy-id -i ~/.ssh/rhel_lab_ed25519.pub administrator@192.168.253.21  
ssh-copy-id -i ~/.ssh/rhel_lab_ed25519.pub administrator@192.168.253.22
```

Test the Key

```
ssh -i ~/.ssh/rhel_lab_ed25519 administrator@192.168.253.21  
exit  
ssh -i ~/.ssh/rhel_lab_ed25519 administrator@192.168.253.22  
exit
```

Why Keys Are Valuable

Passwords can be reused, mistyped, or guessed. A protected SSH key is designed for secure automation. The private key stays on ansible-control and should never be copied to the managed servers.

Phase 5 Create the Ansible Project

Create the Project Folders

```
sudo mkdir -p /ansible-beginner-lab/playbooks
sudo chown -R testuser:testuser /ansible-beginner-lab
cd /ansible-beginner-lab
```

Create the Inventory

```
nano inventory.ini
[web_servers]
rhel-web1 ansible_host=192.168.253.21

[file_servers]
rhel-file1 ansible_host=192.168.253.22

[rhel_servers:children]
web_servers
file_servers

[rhel_servers:vars]
ansible_user=administrator
ansible_ssh_private_key_file=/home/testuser/.ssh/rhel_lab_ed25519
```

Save in Nano by pressing Ctrl+O, Enter, and Ctrl+X.

What the Inventory Does

The inventory is Ansible's address book. It does not configure the servers. It identifies the target systems, their IP addresses, the login account, and the private key Ansible should use.

Validate the Inventory

```
ansible-inventory -i inventory.ini --graph
```

Phase 6 Test Ansible

```
ansible rhel_servers -i inventory.ini -m ansible.builtin.ping
```

Expected Result

```
rhel-web1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}

rhel-file1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Why This Test Is More Than Ping

The Ansible ping module proves that Ansible can find each server, connect through SSH, authenticate, start Python, execute a module, and return a result.

1. Ansible reads inventory.ini.
2. It connects to each address through SSH.
3. It uses the administrator account and SSH key.
4. It starts Python on the managed server.
5. It runs the ping module and receives pong.

Phase 7 Create the Website Playbook

The playbook is the reusable work order. It describes the result you want on both managed servers.

Install the Firewall Collection

```
ansible-galaxy collection install ansible.posix
```

Create the Playbook

```
nano playbooks/install_web.yml
```

Enter the following content. YAML uses spaces to show which lines belong together. Do not use the Tab key.

```
---
- name: Install a website on both Linux servers
  hosts: rhel_servers
  become: true
  gather_facts: true

  tasks:
    - name: Install Apache and firewalld
      ansible.builtin.dnf:
        name:
          - httpd
          - firewalld
        state: present

    - name: Create a customized home page
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"
        backup: true
        content: |
          <!DOCTYPE html>
          <html lang="en">
          <head>
            <meta charset="UTF-8">
            <title>{{ inventory_hostname }}</title>
          </head>
```

```
<body style="font-family: Arial; text-align: center; margin-top: 80px;">
  <h1>Website deployed successfully!</h1>
  <h2>Server: {{ inventory_hostname }}</h2>
  <p>Hostname: {{ ansible_facts['hostname'] }}</p>
  <p>Operating system: {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}</p>
  <p>This website was installed and configured automatically with Ansible.</p>
</body>
</html>
```

```
- name: Start and enable firewalld
```

```
  ansible.builtin.systemd:
```

```
    name: firewalld
```

```
    state: started
```

```
    enabled: true
```

```
- name: Allow web traffic through the firewall
```

```
  ansible.posix.firewalld:
```

```
    service: http
```

```
    permanent: true
```

```
    immediate: true
```

```
    state: enabled
```

```
- name: Start and enable Apache
```

```
  ansible.builtin.systemd:
```

```
    name: httpd
```

```
    state: started
```

```
    enabled: true
```

Phase 8 Understand the Playbook

Playbook section	What it does	Everyday explanation
hosts	Selects rhel_servers	Chooses both destinations from the address book
become	Uses sudo privileges	Receives permission to perform administrator work
gather_facts	Collects system details	Learns each server's name and OS version
dnf	Installs httpd and firewalld	Gets the web-server and firewall software
copy	Creates index.html	Places the custom homepage on each server
firewalld	Allows HTTP traffic	Opens the security gate for website visitors
systemd	Starts and enables Apache	Turns the website on now and after reboot

Why the Pages Are Different

The same template is delivered to both servers, but Ansible replaces variables with the correct information for each system.

```
{{ inventory_hostname }}
{{ ansible_facts['hostname'] }}
{{ ansible_facts['distribution_version'] }}
```

Why backup true Is Included

If index.html already exists, Ansible saves a backup before replacing it. That makes the lesson safer and demonstrates responsible configuration management.

Phase 9 Inspect Before Deploying

Check YAML Structure

```
ansible-playbook -i inventory.ini playbooks/install_web.yml --syntax-check
```

Expected result:

```
playbook: playbooks/install_web.yml
```

Test Administrator Privileges

```
ansible rhel_servers -i inventory.ini -b -K -m ansible.builtin.command -a "whoami"
```

Both servers should return root.

Test Software Repositories

```
ansible rhel_servers -i inventory.ini -b -K -m ansible.builtin.command -a "dnf repolist"
```

Both servers should show enabled repositories. Stop and repair the NAT adapter if either server reports a repository or network error.

Phase 10 Deploy Both Websites

```
ansible-playbook -i inventory.ini playbooks/install_web.yml --diff -K
```

Enter the administrator sudo password when requested.

What the Command Means

Command part	Meaning
ansible-playbook	Starts the automation engine
-i inventory.ini	Supplies the server address book
playbooks/install_web.yml	Supplies the work instructions
--diff	Shows configuration differences
-K	Requests the sudo password

Read the Completion Report

```
PLAY RECAP
rhel-file1 ok=6 changed=4 unreachable=0 failed=0
rhel-web1  ok=6 changed=4 unreachable=0 failed=0
```

Result	Meaning
ok	Tasks completed successfully
changed	Items Ansible created or corrected
unreachable=0	Ansible reached every server
failed=0	No task failed

Phase 11 View the Proof

Open Firefox and visit the first server:

```
http://192.168.253.21
```

The page should identify rhel-web1.

Open a second tab and visit:

```
http://192.168.253.22
```

The page should identify rhel-file1.

About Not Secure: The browser displays this message because the beginner lab uses HTTP rather than encrypted HTTPS. It does not mean the Ansible deployment failed.

Verify from the Command Line

```
curl http://192.168.253.21  
curl http://192.168.253.22
```

Verify Apache on Both Servers

```
ansible rhel_servers -i inventory.ini -m ansible.builtin.command -a "systemctl is-active httpd"  
ansible rhel_servers -i inventory.ini -m ansible.builtin.command -a "systemctl is-enabled httpd"
```

Both servers should report active and enabled.

Phase 12 Prove the Automation Is Repeatable

```
ansible-playbook -i inventory.ini playbooks/install_web.yml -K
```

The second run should report changed=0, unreachable=0, and failed=0.

What Idempotency Means

Idempotency means Ansible checks the desired result before making a change. If the servers already match the playbook, Ansible leaves them alone.

Think of a thermostat. You choose the desired temperature. The thermostat checks the room and acts only when the current temperature does not match the requested temperature. Ansible follows a similar idea for computer configuration.

Final Completion Checklist

- Ansible ping returns pong from both servers.
- The play recap reports unreachable=0 and failed=0.
- Apache reports active and enabled on both servers.
- The .21 website identifies rhel-web1.
- The .22 website identifies rhel-file1.
- The second playbook run reports changed=0.

Troubleshooting Guide

Problem	Likely reason	What to check
Permission denied	SSH account, password, or key problem	Test SSH manually and copy the public key again
Unreachable	Wrong address or network adapter	Run ping and ip -br address
No package httpd available	Repository or NAT problem	Run dnf repolist and check Adapter 1
Missing sudo password	The command omitted -K	Run the playbook with -K
Module ansible.posix.firewalld not found	Collection is missing	Install ansible.posix with ansible-galaxy
Website does not open	Service or firewall issue	Check httpd, port 80, and firewall services
YAML syntax error	Indentation or spelling problem	Use spaces and run --syntax-check

Useful Diagnostic Commands

```
ip -br address
systemctl is-active sshd
systemctl is-active httpd
systemctl is-enabled httpd
sudo ss -tlnp | grep :80
sudo firewall-cmd --list-services
```

Why Ansible Is Valuable

Without automation, an administrator signs into every server and repeats the same work manually. Repetition consumes time and creates opportunities for mistakes. Ansible replaces that repetition with one reviewed, reusable work order.

Manual administration	Ansible automation
Log into every server	Work from one control computer
Repeat every command	Write the instructions once
Settings may become inconsistent	Apply the same standard everywhere
Track results manually	Receive one completion report
Repeat the process later	Reuse the same playbook
More servers require more effort	Manage many servers together

How to Explain the Lab

I used one central computer to send a reusable digital work order to two separate Linux servers. Ansible connected securely, installed the web-server software, created customized pages, configured the firewall, started the services, and reported whether every task succeeded.

The two browser tabs are the visible result. The inventory, YAML playbook, service checks, and play recap are the technical evidence behind that result.

One Sentence Summary

Ansible lets an organization use one approved set of instructions to configure many computers automatically, securely, and consistently.

Student Presentation Script

Use this short script when presenting the project to a nontechnical audience:

I have two independent Linux servers. Normally, I would sign into each server and repeat the installation manually. Instead, I used Ansible to send one reusable work order from a central management computer.

Ansible securely connected to both systems, installed the web-server software, created each webpage, started the services, and configured the firewall.

These two browser tabs show the final result. Each address connects to a different server, and each page displays the identity of that server.

Ansible saves time, reduces human error, keeps systems consistent, and produces a report showing whether every task succeeded.

Task Completed Statement

I built a three-machine Linux automation environment and used an Ansible YAML playbook to deploy Apache websites to two managed servers. The playbook installed the required software, generated server-specific webpages, configured firewall access, enabled the services, and verified the results. A second run confirmed that the deployment was repeatable and made no unnecessary changes.